Decentralized Electronic Health Records using Hyperledger Fabric & IPFS

Blockchain-based EHR with Consent Management and Aggregate Signature Verification

MTech Minor Project · CSE · NIT Warangal

Problem Statement

Traditional EHR systems suffer from four fundamental problems:

- centralized data silos where patient records are locked within a single hospital's database and cannot be securely shared across providers;
- lack of patient agency — patients have no control over who accesses their medical data
- absence of a tamper-evident audit trail, making it impossible to prove after-the-fact that data was not altered; and
- scalability bottlenecks from storing large clinical payloads (full medical records, lab reports) on-chain.

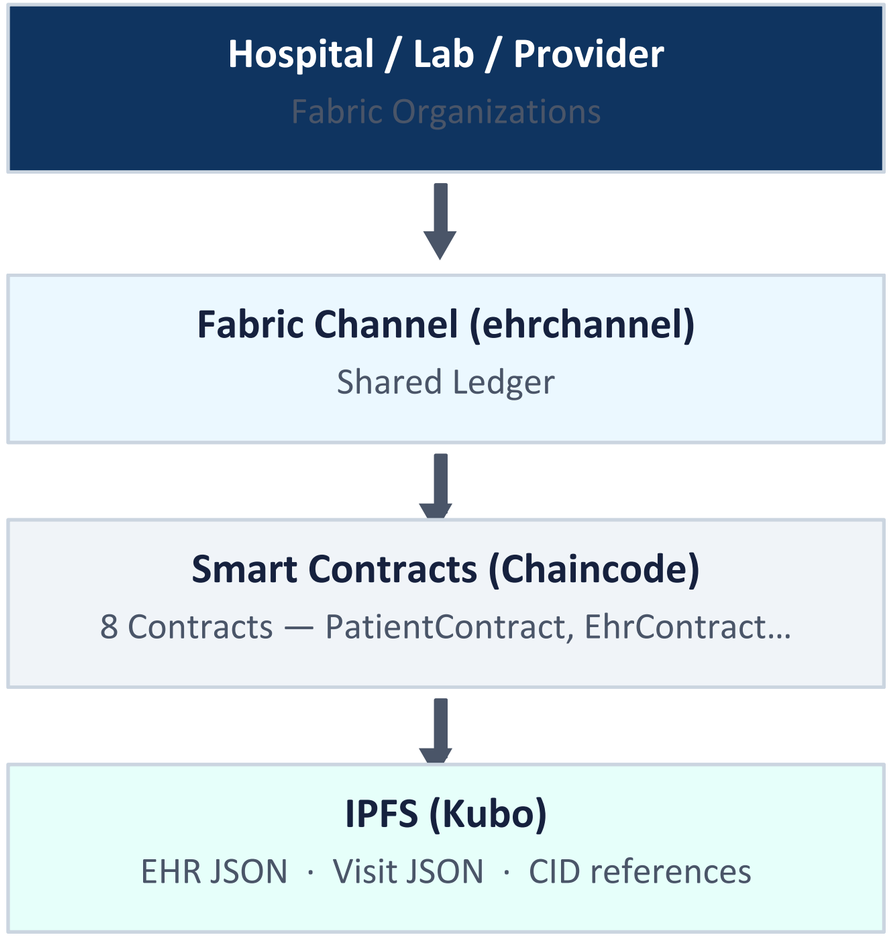
Our Solution — Hybrid Blockchain + IPFS Architecture

Hyperledger Fabric (Permissioned Blockchain)

- Multi-org permissioned network — only verified actors participate
- Immutable ledger records every state transition (patient reg, visit lifecycle, consent grants)
- Role attributes embedded in X.509 certificates — cryptographically enforced RBAC
- On-chain stores: patient records, visit status, EHR CIDs, access grants, audit log

IPFS via Kubo (Off-chain Storage)

- Clinical payloads (EHR JSON, Visit JSON) stored off-chain
- Content-addressed CIDs — tampering changes the CID, invalidating the on-chain reference
- Every update creates a new CID — immutable version history



Overall Network Architecture — 3 Organizations

Organization	MSP ID	Role	Peer(s)
Hospital	HospitalMSP	Primary care provider	peer0, peer1, peer2
Diagnostics Lab	DiagnosticsMSP	Lab & radiology	peer0.diagnostic
Provider / Insurance	ProviderMSP	Billing, claims, insurance	peer0.provider

Single Channel: ehrchannel · All orgs hold full ledger replica · Raft CFT consensus

```
NETWORK STATUS

Containers:
✓ docker-peer2.hospital.example.com-1 Up 30 seconds
✓ docker-peer1.hospital.example.com-1 Up 30 seconds
✓ docker-peer0.hospital.example.com-1 Up 31 seconds
✓ docker-orderer.example.com-1        Up 31 seconds
✓ docker-peer0.provider.example.com-1 Up 31 seconds
✓ ipfs.ehr.local      Up 31 seconds (healthy)
✓ docker-peer0.diagnostic.example.com-1       Up 31 seconds
✓ docker-ca-orderer-1 Up 44 seconds
✓ docker-ca-hospital-1        Up 44 seconds
✓ docker-ca-provider-1        Up 44 seconds
✓ docker-ca-diagnostics-1     Up 44 seconds

Channel: ehrchannel

Peers on channel:
✓ peer0.hospital    :7051  → Auth/Reception
✓ peer1.hospital    :9051  → Doctor
✓ peer2.hospital    :10051 → Nurse/Pharmacist
✓ peer0.diagnostic :8051   → Lab
✓ peer0.provider    :11051 → Insurance

IPFS Node:
✓ ipfs.ehr.local
    API:     http://localhost:5001
    Gateway: http://localhost:8090
```

Hospital Organization — Peer-to-Role Mapping

peer0 :7051

Roles

- **Receptionist**
- **Admin**

API Service

- peer0-api :3000

Routes

/patients
/visits (open/assign/discharge)
/auth /staff

peer1 :8051

Roles

- **Doctor**

API Service

- peer1-api :3001

Routes

/doctor/visits
/doctor/visits/:id/diagnosis
/doctor/visits/:id/prescription
/doctor/visits/:id/finalize

peer2 :9051

Roles

- **Nurse**
- **Pharmacist**
- **MedRecord Officer**

API Service

- peer2-api :3002

Routes

/nurse/visits/:id/vitals
/nurse/visits/:id/carenote
/pharmacist/:id/dispense
/records/:id/finalize

Role separation is cryptographically enforced — each API only loads wallet identities for its peer's roles

Fabric CA — Register & Enroll with Role Attributes

1 Register

Admin tells CA "this identity exists + these attributes". No cert issued yet.

2 Enroll

Identity holder presents credentials → receives signed X.509 cert + private key. Private key never leaves the machine.

3 Attribute Embed

"role=doctor:ecert" flag causes the role string to be embedded inside the signed certificate.

4 Chaincode Read

ctx.clientIdentity.getAttributeValue('role') reads role directly from cert — no DB lookup, no trust assumption.

Certificate Hierarchy

- Root CA (ca.hospital.example.com)
  - TLS CA — for peer TLS
  - Signing CA
    - peer0 / peer1 / peer2 (peer certs)
    - receptionist (role=receptionist:ecert)
    - doctor (role=doctor:ecert)
    - nurse (role=nurse:ecert)
    - patientService (role=patientService:ecert)

Fabric CA — Register & Enroll with Role Attributes

```
# Step 1: Register (done by CA admin once)
fabric-ca-client register \
  --id.name nurse \
  --id.secret nursepw \
  --id.type client \
  --id.attrs "role=nurse:ecert" \
  --tls.certfiles "$CADIR/tls-cert.pem" \
  -u https://ca.hospital.example.com:7054

# Step 2: Enroll (done by the nurse's server at startup)
fabric-ca-client enroll \
  --id.name nurse \
  --id.secret nursepw \
  --tls.certfiles "$CADIR/tls-cert.pem" \
  -u https://ca.hospital.example.com:7054 \
  -M "$USERS/nurse/msp"
```

```
==========================================
 ALL IDENTITIES ENROLLED SUCCESSFULLY
==========================================

Hospital peers enrolled:
  peer0.hospital  → Auth / Reception peer  (port 7051)
  peer1.hospital  → Doctor peer             (port 9051)
  peer2.hospital  → Nurse + Pharmacist peer (port 10051)

Role attributes embedded in certs:
  receptionist     → role=receptionist    [connects to peer0]
  doctor           → role=doctor          [connects to peer1]
  nurse            → role=nurse           [connects to peer2]
  pharmacist       → role=pharmacist      [connects to peer2]
  medrecordofficer → role=medrecordofficer [connects to peer1]
  hospitaladmin    → role=admin           [connects to peer0]
  patientService   → role=patientService  [used by patient-api — signs on behalf of patients]
  labreceptionist  → role=labreceptionist
  labtechnician    → role=labtechnician
  labsupervisor    → role=labsupervisor
  radiologist      → role=radiologist
  billingofficer   → role=billingofficer
  claimsauditor    → role=claimsauditor
  insuranceofficer → role=insuranceofficer

[OK]    All identities enrolled
```

Channel Creation via OSN Admin API (Fabric 2.3+)

Why OSN Admin API? The legacy approach required ordering admin to be a channel member. OSN Admin API cleanly separates ordering service administration from channel membership.

1 configtxgen

Generate genesis block with configtx.yaml listing all 3 org MSPs + orderer definition

2 osnadmin channel join

POST genesis block to orderer OSN Admin API over mutual TLS — channel created

3 peer channel join

Each peer (all 5) joins the channel using the genesis block

4 Anchor Peers

Set anchor peers per org so cross-org gossip protocol can discover peers

```
osnadmin channel join \
  --channelID ehrchannel \
  --config-block ./channel-artifacts/ehrchannel.block \
  -o localhost:7053 \
  --ca-file "$ORDERER_CA" \
  --client-cert "$ADMIN_TLS_CERT" \
  --client-key "$ADMIN_TLS_KEY"
```

```
peer channel join \
  -b ./channel-artifacts/ehrchannel.block \
  --tls --cafile "$ORDERER_CA"
```

Result: All 5 peers on ehrchannel · All 3 orgs hold full ledger replica

Chaincode Deployment — Fabric 2.x Lifecycle

Fabric 2.x requires multi-org approval before commit — no single org can unilaterally upgrade chaincode logic.

Package

`peer lifecycle chaincode package`

Creates .tar.gz from ./ehr-chaincode-v3 (Node.js)

Install

`chaincode install`

Installed on all 5 peers; each returns a package ID

Approve

`approveformyorg`

Each org admin approves — name, version, sequence, package ID

Check

`checkcommitreadiness`

Verify all orgs have approved before committing

Commit

`chaincode commit`

Committed to channel — now live

Invoke

`peer chaincode invoke`

Chaincode invoked; endorsement policy validated

Chaincode Architecture — 8 Smart Contracts

PatientContract

Register, GetPatient, ListAll, SetEhrCID

EhrContract

InitEHR, UpdateEHRCID, GetCurrentCID, GetCIDHistory, GetBlockHistory

VisitContract

OpenVisit, AssignDoctor/Nurse, FinalizeVisit, FinalizeRecord, DischargePatient, ListAllVisits

AccessContract

GrantAccess, RevokeAccess, RequestAccess, ApproveRequest, RejectRequest, GetActiveGrants, GetAuditLog

ClinicalContract

RecordVitals, AddCareNote, UpdatePrescription

ForwardContract

Forward visit between roles with audit trail entries

LabContract

CreateLabRequest, AcknowledgeRequest, SubmitResults, ApproveResults

ClaimsContract

SubmitClaim, AuditClaim, ProcessClaim (Approve/Reject)

Shared accessControl.js · getCallerIdentity() · requireRole() · assertReadAccess() · assertEHRWriteAccess()

Ledger Key Scheme & On-Chain vs IPFS Data Split

Ledger Key Scheme

`PATIENT:<patientId>`

Demographics, visitIds, ehrCID, visitCount

`EHR:<patientId>`

currentCID, cidHistory[], createdAt, updatedAt

`VISIT:<visitId>`

status, assignedDoctor/Nurse, visitCID, cidHistory[], claimFields

`ACCESS:<patientId>`

grants[], requests[], auditLog[]

`CLAIM:<visitId>`

claimId, amount, status, auditedBy, processedBy

On-Chain vs IPFS Boundary

On-Chain vs IPFS Boundary:

Data	Location	Reason
Patient demographics (name, age, etc.)	**Both** (on-chain master + in EHR JSON)	On-chain for fast lookup; IPFS for rich context
EHR current CID + cidHistory	**On-chain**	Integrity anchor — CID is tamper-evident
Actual EHR JSON (allergies, conditions, etc.)	**IPFS**	Large, variable-length; off-chain avoids ledger bloat
Visit status, assignedDoctor, claimStatus	**On-chain**	Small fields, needed for chaincode logic and endorsement
Visit clinical content (vitals, notes, prescriptions)	**IPFS**	Large payload; fetched on demand
Access grants, audit log	**On-chain**	Critical for verifiable consent — must be in ledger

CID = SHA-256 hash of content. Any IPFS file tampering → CID changes → on-chain reference invalid.

Two-Template Design — EHR vs Visit: Ownership Boundary

Core Principle: EHR is patient-owned (sensitive, few actors). Visit is hospital-owned (operational, broad access).

EHR Document (Patient-Owned)

- Scope: Lifelong — persists across ALL visits
- Fields: allergies, chronicConditions, ongoingMedications, surgicalHistory, familyHistory, immunizations, emergencyContact, demographics, lifestyle.
- Writers: Doctor + Nurse (ONLY with patient consent grant), Admin (demographics only)
- Readers: Doctor + Nurse require explicit on-chain grant for the "ehr" section
- Updated: Only when patient consents AND clinically needed
- Access: Patient controls via AccessContract grants

Visit Document (Hospital-Owned)

- Scope: Single encounter — closed at discharge
- Fields: chiefComplaint, forwardingLog, diagnosisNotes, finalDiagnosis, prescriptions[], vitals, careNotes, labRequests[], medicationDetails, dischargeNotes
- Writers: ALL clinical staff — doctor, nurse, pharmacist, medrecordofficer, lab staff, billing
- Readers: ALL recognized staff roles (broad access)
- Updated: Every clinical action during the encounter
- Final EHR Update: Clinician promotes findings to EHR at end of visit (with patient consent)

Two-Template Rationale — Design Decision Table

```
{
 "_type": "EHR",
 "_version": 1,
 "patientId": "PAT-001",
 "createdAt": "<ISO timestamp>",
 "updatedAt": "<ISO timestamp>",
 "updatedBy": "system",

 "demographics": {
  "name": "",
  "age": "",
  "gender": "",
  "bloodGroup": "",
  "contact": "",
  "address": "",
  "dob": ""
 },

 "allergies": [],
 // entry: { substance, reaction, severity, addedBy, addedAt }

 "chronicConditions": [],
 // entry: { condition, diagnosedAt, status, notes, addedBy, addedAt }

 "ongoingMedications": [],
 // entry: { name, dose, frequency, since, prescribedBy, addedAt }

 "surgicalHistory": [],
 // entry: { procedure, date, hospital, surgeon, notes, addedBy, addedAt }

 "familyHistory": [],
 // entry: { relation, condition, notes, addedBy, addedAt }

 "immunizations": [],
 // entry: { vaccine, date, dose, administeredBy, addedAt }

 "emergencyContact": {
 "name": "",
 "relation": "",
 "phone": "",
 "address": ""
 },

 "lifestyle": {
 "smoking": "", // 'never' | 'former' | 'current'
 "alcohol": "", // 'none' | 'occasional' | 'regular'
 "exercise": "", // 'sedentary' | 'light' | 'moderate' | 'active'
 "diet": ""
 }

}
```

http://bafkreib25kdosdar7bzjn5il2abhkkysatejetqzn3ahujqqcencoirmee.ipfs.localhost:8090

JSON | Raw Data | Headers

Save | Copy | Pretty Print


```
{
  "_type": "EHR",
  "_version": 1,
  "patientId": "PAT-2026-001",
  "createdAt": "2026-04-20T19:12:50.992Z",
  "updatedAt": "2026-04-20T19:33:57.733Z",
  "updatedBy": "patient:PAT-2026-001",
  "demographics": {
    "name": "Rahul Mehta",
    "age": 23,
    "gender": "Male",
    "bloodGroup": "O+",
    "contact": "9123788550",
    "address": "123 mg road",
    "dob": ""
  },
  "allergies": [
    {
      "allergy": "Aspirin",
      "severity": "High",
      "reaction": "GI bleeding, platelet dysfunction"
    }
  ],
  "chronicConditions": [],
  "ongoingMedications": [],
  "surgicalHistory": [],
  "familyHistory": [],
  "immunizations": [],
  "emergencyContact": {
    "name": "",
    "relation": "",
    "phone": "",
    "address": ""
  },
  "medicalHistory": [
    {
      "text": "Dengue Fever — NS1 Positive. Platelet nadir 85,000. Managed with IV fluids and supportive care. Full recovery.",
      "sourceType": "MANUAL",
      "attributes": {
        "diagnosis": "Dengue Fever (NS1 Antigen Positive)",
        "treatment": "IV fluids, Paracetamol, ORS, Pantoprazole",
        "doctor": "dr_arjun"
      }
    },
    {
      "text": "HE\nfip— Res ax -\n= p2\n= LLL EE\nLABORATORY REPORT\nName: John Doe\nReport Date: 12/05/2024\nMedical Center: Apex Health Partners\nFindings: Mild iron deficiency anemia\nMedication® Ferrous sulfate
325mg daily\nDoctor: Dr. Sarah Johnson\nHemoglobin 44 Apct 405-90\nHematocrit 52.4 420-45\nMean Corpuscula® Volume M0 (80-24\nMean error BW) 90.7 Wigner\nHematicin® 565.5pi0 Nore\npoltiflanic 18 Poclr\nElozpherinaten®
031 Mon\nHematocrit (HV) 009 No\nNe 7h 049 A\noi N",
      "sourceType": "ocr",
      "sourceCid": "bafkreig7th4ufprgbtthnuohr3in2mza3xifrioswc7k7eg22i2iixtzfa",
```

Two-Template Rationale — Design Decision Table

```
{
 "_type": "VISIT",
 "_version": 1,
 "visitId": "PAT-001-V1",
 "patientId": "PAT-001",
 "createdAt": "<ISO timestamp>",
 "updatedAt": "<ISO timestamp>",

 "chiefComplaint": "",

 "forwardingLog": [
  {
   "action": "VISIT_OPENED",
   "from": "<receptionistId>",
   "fromRole": "receptionist",
   "to": "",
   "toRole": "",
   "notes": "<chiefComplaint>",
   "timestamp": "<ISO timestamp>"
  }
 ],

 // ── Doctor section ──────────────────────────
 "assignedDoctor": "",
 "diagnosisNotes": "",     // working notes, updated multiple times
 "finalDiagnosis": "",     // set when doctor calls FinalizeVisit
 "finalizedBy": "",
 "finalizedAt": "",
 // ── Discharge section ───────────────────────
 "dischargeNotes": "",
 "dischargedBy": "",
 "dischargedAt": ""

 "prescriptions": [],
 // entry: { version, medications: [], instructions, prescribedBy, prescribedAt }

 // ── Nurse section ───────────────────────────
 "assignedNurse": "",

 "vitals": null,
 // { bloodPressure, temperature, pulse, weight, height, oxygenSat, recordedBy, recordedAt }

 "careNotes": [],
 // entry: { note, recordedBy, recordedAt }

 // ── Lab section ─────────────────────────────
 "labRequests": [],
 // entry: {
 // labRequestId, tests: [], instructions, requestedBy, requestedAt,
 // status: REQUESTED | ACKNOWLEDGED | COMPLETED | APPROVED,
 // acknowledgedBy, acknowledgedAt,
 // results: {}, resultsHash, submittedBy, submittedAt,
 // approvedBy, approvedAt
 // }

 // ── Pharmacy section ────────────────────────
 "medicationDetails": "",
 "medicationDispensedBy": "",
 "medicationDispensedAt": "",

 // ── Medical Records section ─────────────────
 "recordFinalizedBy": "",
 "recordFinalizedAt": "",

}
```

```
{
  "_type": "VISIT",
  "_version": 1,
  "visitId": "PAT-2026-001-V1",
  "patientId": "PAT-2026-001",
  "createdAt": "2026-04-20T19:13:35.728Z",
  "updatedAt": "2026-04-20T19:25:17.003Z",
  "chiefComplaint": "Persistent fever for 3 days with severe headache and body aches",
  "forwardingLog": [
    {
      "action": "VISIT_OPENED",
      "from": "receptionist",
      "fromRole": "receptionist",
      "to": "",
      "toRole": "",
      "notes": "Persistent fever for 3 days with severe headache and body aches",
      "timestamp": "2026-04-20T19:13:35.729Z"
    },
    {
      "action": "DOCTOR_ASSIGNED",
      "from": "receptionist",
      "fromRole": "receptionist",
      "to": "doctor",
      "toRole": "doctor",
      "notes": "",
      "timestamp": "2026-04-20T19:16:12.476Z"
    },
    {
      "action": "NURSE_ASSIGNED",
      "from": "receptionist",
      "fromRole": "receptionist",
      "to": "nurse",
      "toRole": "nurse",
      "notes": "",
      "timestamp": "2026-04-20T19:16:22.893Z"
    },
    {
      "action": "DIAGNOSIS_NOTES_UPDATED",
      "from": "doctor",
      "fromRole": "doctor",
      "to": "",
      "toRole": "",
      "notes": "Patient presents with high-grade fever (102°F), retro-orbital headache, myalgia, and mild rash on trunk. Clinical picture Notes consistent with Dengue Fever. Awaiting NS1 antigen and
CBC confirmation.",
      "timestamp": "2026-04-20T19:18:35.689Z"
    },
    {
```

EHR Version History & Three-Layer Audit Trail

On-Chain EHR Record — cidHistory[]

bafyrei…001

by: receptionist · section: all

EHR_INITIALISED

bafyrei…002

by: doctor · section: allergies

Allergy to penicillin documented

bafyrei…003

by: doctor · section: chronicConditions

Type 2 diabetes diagnosed

bafyrei…004

by: nurse · section: immunizations

Immunization record updated

Three-Layer Audit Trail

1

Blockchain TX History

ctx.stub.getHistoryForKey() — every putState call has a txId assigned by orderer. Tamper-evident, permanent. EhrContract:GetEHRBlockHistory, VisitContract:GetVisitHistory

2

CID History (Content Versioning)

cidHistory[] in on-chain record — section-level granularity, human-readable reason, updatedBy. Historical CIDs can still be fetched from IPFS (pinned).

3

Access Audit Log (Consent Events)

ACCESS:<patientId>.auditLog — GRANT, REVOKE, REQUEST, READ events. Who accessed what and when. Immutable on-chain. AccessContract:GetAuditLog

Backend Microservices Architecture — 6 Node.js Services

peer0-api :3000

Roles: Receptionist
Admin

```
/patients  /visits
/auth  /staff
```

peer1-api :3001

Roles: Doctor

```
/doctor/visits
/doctor/visits/:id/diagnosis
/prescription  /ehr  /finalize
```

peer2-api :3002

Roles: Nurse
Pharmacist
MedRecord

```
/nurse/visits/:id/vitals
/nurse/:id/carenote
/pharmacist/:id/dispense
/records/:id/finalize
```

patient-api :3003

Roles: Patient
(patientService acct)

```
/auth  /ehr  /visits
/access  /signatures
/access/requests
```

extorg-api :3004

Roles: Lab Staff
Provider/Billing

```
/lab  /claims
```

ipfs-service :3005

Roles: Internal
(all backends)

```
/pin  /fetch/:cid
/ehr/init  /visit/init
```

Two-Layer Security Model & Gateway Manager

Two-Layer Security

Layer 1: JWT Verification (API Middleware)

Every request must carry a valid JWT. Middleware verifies signature, extracts {userId, role, mspId, peer}. requireRole() middleware gates API routes by role before the request reaches chaincode.

Layer 2: Certificate-Embedded Role (Chaincode)

Transaction signed with enrollee's X.509 cert. Chaincode reads role attribute directly from cert using getAttributeValue('role'). requireRole(ctx, 'doctor') enforced cryptographically — cannot be bypassed by API code.

Result: Even if API is compromised, chaincode rejects wrong-cert requests

GatewayManager.js — Per-Request Fabric Connection

1. Read role from JWT payload
2. peerRouter.js → peer address + MSP path for that role
3. Load enrolled X.509 identity from filesystem wallet
4. Create Fabric Gateway connection (mutual TLS)
5. Return contract handle: ehrchannel → ehrcc
6. All chaincode calls signed with role's cert automatically

Design Choice — Patient Service & Off-Chain SQLite

Why NOT enroll every patient as a Fabric identity in the CA?

CA Scalability

Fabric CA is for organizational identities — staff, peers, admins. 10,000s of patients overwhelm CA and complicate CRL management.

Key Management

Real patients cannot manage crypto private keys. No recovery if key is lost. Patients expect username/password auth.

Certificate Binding

patientId-in-cert requires one-to-one patient↔cert mapping. Multi-device scenarios require complex key distribution.

Our Solution: patientService Pattern

Patient (Browser)

POST /auth/login
{ patientId, password }

→

patient-api

Verify against SQLite (bcrypt)
Issue JWT { patientId }

→

patient-api (JWT verified)

POST /access/grant
{ granteeId: "doctor"
sections: ["ehr"] }

→

Chaincode (patientService cert)

AccessContract: GrantAccess()
requireRole("patientService") ✓
Grant on-chain

`SQLite stores: patientId (TEXT PK), passwordHash (bcrypt), email, phone — NO clinical data`

Patient Journey — Complete Flow from Arrival to Discharge

RECEPTION
peer0-api

① Register patient + pin EHR to IPFS
② Open Visit + pin Visit JSON
③ Assign doctor → WITH_DOCTOR

DOCTOR
peer1-api

④ Read Visit JSON from IPFS
⑤ Update diagnosis + prescriptions
⑥ Read/Update EHR (if patient consent)
⑦ Assign nurse → FinalizeVisit

NURSE
peer2-api

⑧ Record vitals
⑨ Add care notes
⑩ View prescriptions

LAB
extorg-api

⑪ Acknowledge request
⑫ Submit results + hash
⑬ Supervisor approves

PHARMACY
peer2-api

⑭ View prescription
⑮ Dispense + record

MED RECORDS
peer2-api

⑯ Review + FinalizeRecord
Status → RECORD_FINALIZED

BILLING
extorg-api

⑰ Submit claim
⑱ Audit + Approve/Reject

DISCHARGE
peer0-api

⑲ Admin: DischargePatient
Status → DISCHARGED

Patient Portal (parallel): Track live visit status · View EHR · Grant/Revoke access · Approve access requests · Verify aggregate signatures

Novel Contributions

① Consent Management via X.509 Role Attributes

② Aggregate Transcation Verification

Novel Contribution 1 — Cryptographic Consent Management

Problem: Existing systems enforce access control at the application layer — not verifiable, bypassable by admin, no cryptographic proof.

Our Approach: Two Interlocking Mechanisms

Mechanism 1 — Role in X.509 Certificate

- Role attribute embedded by Fabric CA at enrollment: "role=doctor:ecert"
- Attribute is part of the CA-signed certificate — cannot be forged
- Chaincode reads: ctx.clientIdentity.getAttributeValue("role")
- No database lookup — role proven by CA signature alone
- Who you are is proven cryptographically

```
Certificate Subject: CN=doctor, OU=client, O=hospital
X.509v3 Extension - Fabric Custom Attributes:
  role: doctor
  peer: peer1
```

Mechanism 2 — On-Chain Consent Grant

- Patient stores explicit grants in ACCESS:<patientId>
- Sections: ehr | visits | prescriptions | labResults | all
- Each grant: granteeId, sections[], expiresAt, revoked
- assertReadAccess() checks on-chain grant in chaincode
- Doctor + Nurse in EHR_CONSENT_REQUIRED_ROLES — must have explicit grant even though they are "staff"

```
const role = ctx.clientIdentity.getAttributeValue('role'); // → "doctor"
```

Fabric CA — Register & Enroll with Role Attributes

```
# Step 1: Register (done by CA admin once)
fabric-ca-client register \
  --id.name nurse \
  --id.secret nursepw \
  --id.type client \
  --id.attrs "role=nurse:ecert" \
  --tls.certfiles "$CADIR/tls-cert.pem" \
  -u https://ca.hospital.example.com:7054

# Step 2: Enroll (done by the nurse's server at startup)
fabric-ca-client enroll \
  --id.name nurse \
  --id.secret nursepw \
  --tls.certfiles "$CADIR/tls-cert.pem" \
  -u https://ca.hospital.example.com:7054 \
  -M "$USERS/nurse/msp"
```

Consent Management — Access Request & Grant Flow

Doctor (peer1-api)

AccessContract: RequestAccess()
role: doctor

→

On-Chain ACCESS Record

status: PENDING
requesterId: "doctor"
sections: ["ehr","visits"]

→

Patient Portal (patient-api)

GET /access/requests
Sees pending request

↓

Patient Approves (or Rejects)

POST /access/requests /:requestId/approve
{ expiresAt: "..." }

→

AccessContract ApproveRequest

PENDING → APPROVED
New grant created:
grants[] updated

→

Doctor can now read EHR

GetCurrentCID succeeds
assertReadAccess ✓
Grant found & valid

The patient controls access via the AccessContract on-chain. The consent record for a patient (ACCESS:<patientId>) stores:


```
{
  "patientId": "PAT-042",
  "grants": [
    {
      "grantId": "PAT-042-G1",
      "granteeId": "doctor",
      "granteeRole": "doctor",
      "sections": ["ehr", "visits"],
      "grantedBy": "patientService",
      "grantedAt": "2026-04-19T10:30:00Z",
      "expiresAt": "2026-06-01T00:00:00Z",
      "revoked": false
    }
  ],
  "requests": [...],
  "auditLog": [...]
}
```

Consent Management — Security Properties & Comparison

Security Properties

- **Identity Authenticity** — role attribute embedded in CA-signed X.509 certificate at enrollment. Cannot be forged — proven by CA signature, not a database field.
- **Access Authorization at Chaincode Level** — assertReadAccess() runs inside chaincode on the peer. API server cannot bypass it — wrong certificate means transaction rejected regardless of application logic.
- **Patient Sovereignty** — Only patientService (called after patient JWT is verified off-chain) can invoke GrantAccess and RevokeAccess. No staff member can grant themselves access to any patient's records.
- **Time-bounded and Instantly Revocable** — Every grant carries expiresAt enforced inside chaincode. RevokeAccess() sets revoked: true on-chain immediately, next access attempt rejected with zero delay, no token invalidation window.
- **Immutable Audit Trail** — Every GRANT, REVOKE, REQUEST, READ event appended to ACCESS:<patientId>.auditLog[] on-chain at time of occurrence. Cannot be deleted or modified by the hospital, staff, or API server.

Novel Contribution 2 — Aggregate Transaction

Problem: Blockchain guarantees on-chain data is immutable. But the actual clinical content lives in IPFS, a patient with limited technical knowledge cannot verify IPFS integrity directly. Also, even on-chain visit records accumulate multiple transactions over time, the patient has no simple way to know if an unexpected transaction was added to their record.

Goal: Give the patient a single verifiable number, a hash that represents the complete history of their record. If that number changes unexpectedly, they know something was added or tampered with.

Novel Contribution 2 — Aggregate Transaction Verification

Purpose: Give the patient a user-friendly, verifiable integrity proof that their medical records have not been tampered with since they were recorded.

Visit Aggregate — Transaction IDs

- GET VisitContract:GetVisitHistory(visitId)
- Extract all txId values from blockchain TX history
- Sort txIds for determinism
- SHA256(sorted txIds concatenated) → aggregateHash
- txIds are assigned by orderer — cannot be forged

```
SHA256( sort([txId1, txId2, txId3,
...]).join("") )
```

```
function computeAggregate(ids) {
  const real = ids.filter(id => id);
  if (!real.length) return null;
  return crypto.createHash('sha256')
    .update([...real].sort().join(''))
    .digest('hex');
}
```

```
function computeEhrAggregate(entries) {
  const cids = entries.map(e => e.cid).filter(Boolean);
  return computeAggregate(cids);
}
```

Aggregate Transcation — Verification Flow & Assessment

```
VISIT 1 (April 2026)
  Patient opens /signatures
  Sees timeline: Visit opened → Doctor assigned → Vitals recorded → Visit
finalized → Discharged
  aggregateHash = "e4f7a2b9c1..."
  Patient screenshots or copies this hash

VISIT 2 (June 2026) — Patient wants to check no one tampered with Visit 1
  Patient opens /signatures/verify/visit/PAT-001-V1?hash=e4f7a2b9c1...
  Backend:
    1. Fetch GetVisitHistory(PAT-001-V1) from chain
    2. Extract txIds
    3. Recompute SHA256(sorted txIds)
    4. Compare against e4f7a2b9c1...
  Response: { valid: true, aggregateHash: "e4f7a2b9c1...", txCount: 7 }

  If valid: false → txCount changed → n  ransaction was added after patient
saved the hash
```

CURRENT WORLD STATE [live — queried from ledger]

▶ PATIENT RECORDS

PATIENT:PAT-2026-001

```
patientId  :  PAT-2026-001
name  :  Rahul Mehta
age  :  23
gender  :  Male
bloodGroup  :  O+
contact  :  9123788550
address  :  123 mg road
visitIds  :
    → PAT-2026-001-V1
visitCount  :  1
ehrCID  :  bafkreicbb6xrkb5ab4vbdjaihdqrtdebviaxrdajmzazbvqbrps3uzbhru
registeredBy  :  receptionist
createdAt  :  2026-04-20T19:12:51.150Z
updatedAt  :  2026-04-20T19:13:35.755Z
```

```
▶ EHR DOCUMENTS

┌─ EHR:PAT-2026-001
│  patientId  :  PAT-2026-001
│  currentCID  :  bafkreib25kdosdar7bzjn5il2abhkkysatejetqzn3ahujqqcencoirmee
│  cidHistory  :
│      → {"cid":"bafkreicbb6xrkb5ab4vbdjaihdqrtdebviaxrdajmzazbvqbrps3uzbhru","updatedBy":"receptionist","updatedAt":"2026-04-20T19:12:53.277Z"
,"reason":"EHR_INITIALISED","section":"all"}
│      → {"cid":"bafkreicg7bjevmz52hbpf5k4lsassroe2wbdfp66pkh5nefku3ipqzbnum","updatedBy":"doctor","updatedAt":"2026-04-20T19:25:53.027Z","reas
on":"Updated by doctor doctor","section":"allergies"}
│      → {"cid":"bafkreiflt56fglnfvnkea6ouqhecvkqq3mdjas5f75otr34fhakzbrjx3a","updatedBy":"doctor","updatedAt":"2026-04-20T19:26:57.565Z","reas
on":"Updated by doctor doctor","section":"medicalHistory"}
│      → {"cid":"bafkreibo4r7lkmrm3nmam2d2a2bxyxgin5732pmmfsfuh735hnomjwgp6e","updatedBy":"nurse","updatedAt":"2026-04-20T19:29:29.023Z","reaso
n":"Updated by nurse nurse","section":"lifestyle"}
│      → {"cid":"bafkreib25kdosdar7bzjn5il2abhkkysatejetqzn3ahujqqcencoirmee","updatedBy":"patientService","updatedAt":"2026-04-20T19:33:57.756
Z","reason":"New medical history entry","section":"medicalHistory"}
│  createdAt  :  2026-04-20T19:12:53.277Z
│  updatedAt  :  2026-04-20T19:33:57.756Z
└──
```

```
▶ VISIT RECORDS

┌─ VISIT:PAT-2026-001-V1
│  visitId  :  PAT-2026-001-V1
│  patientId  :  PAT-2026-001
│  visitNumber  :  1
│  status  :  WITH_DOCTOR
│  assignedDoctor  :  doctor
│  assignedNurse  :  nurse
│  visitCID  :  bafkreiefwp7w7sqc3ihivu24u4zrm2pnvc3rrgxw3s3y2sufk276yvo6cq
│  cidHistory  :
│      → {"cid":"bafkreicltzt5psunp4cyop5ztazntj3qibvxbhsarbbicr2lei3llpx3d4","updatedBy":"receptionist","updatedAt":"2026-04-20T19:13:35.755Z
,"reason":"VISIT_OPENED"}
│      → {"cid":"bafkreia75aou5jwzrxm4ggdoa77fmwulz3lqj73xxsrcdjskt5bvjhtem4","updatedBy":"receptionist","updatedAt":"2026-04-20T19:16:12.503Z
,"reason":"DOCTOR_ASSIGNED"}
│      → {"cid":"bafkreifrmwe7asghwixjs6xpwxce2eix6qvowqc6paapkcqfp7l5u23p5y","updatedBy":"receptionist","updatedAt":"2026-04-20T19:16:22.919Z
,"reason":"NURSE_ASSIGNED"}
│      → {"cid":"bafkreibraubbusn4oevqrpg3jr7fzsz7fsr42kvutlpi64qw2mfstlm7g4","updatedBy":"doctor","updatedAt":"2026-04-20T19:18:35.737Z","rea
on":"DIAGNOSIS_NOTES_UPDATED"}
│      → {"cid":"bafkreicjmzubdcbgwmwmgviy3fmsxrcvt6nfufocp3vqxyw322yzgjk6we","updatedBy":"doctor","updatedAt":"2026-04-20T19:19:37.305Z","rea
on":"FORWARD_TO_NURSE"}
│      → {"cid":"bafkreigdmalfkm5tckvaox2w75qmiqk25yqrtp5gjd7iawwd53c2523mbm","updatedBy":"nurse","updatedAt":"2026-04-20T19:21:18.120Z","reas
n":"VITALS_RECORDED"}
│      → {"cid":"bafkreibtmprlwb7iluzyus34v2fsmwfgkgk4djtqhho33ieupontjoctiq","updatedBy":"nurse","updatedAt":"2026-04-20T19:21:45.613Z","reas
n":"CARE_NOTE_ADDED"}
│      → {"cid":"bafkreiet2jdtr3i5bvtc2uwuspjzpr7xflsmhd7o2zzwx6djn5s5x5jxui","updatedBy":"nurse","updatedAt":"2026-04-20T19:22:43.211Z","reas
n":"FORWARD_TO_DOCTOR"}
│      → {"cid":"bafkreiefwp7w7sqc3ihivu24u4zrm2pnvc3rrgxw3s3y2sufk276yvo6cq","updatedBy":"doctor","updatedAt":"2026-04-20T19:25:17.077Z","rea
on":"PRESCRIPTION_UPDATED"}
│  claimId  :
│  claimAmount  :  0
│  claimSubmittedBy  :
```

```
┌─ Block #0 ──────────────────────────────────────────────────────────────────
  prev_hash : null...
  data_hash : TKEI2rIMq/pioQVy25Hr...
  [Genesis block — channel configuration]
└─────────────────────────────────────────────────────────────────────────────

┌─ Block #10 ─────────────────────────────────────────────────────────────────
  prev_hash : KFiHgG49YNxXYRudDlgd...
  data_hash : 4EKOMVZ91sLt2v5OoiWV...
  txns      : 1

  Tx #0
  TxID      : 61946ffeea80a2baf32909b55ed3b0739473acaae574ec3837062828e968017f
  Time      : 2026-04-19T10:36:06.180Z
  Creator   : HospitalMSP  ()
  Function  : VisitContract:AssignDoctor  a-V1 doctor bafkreihop3gccwnfzg5nzp3om2m2ngud773jnvbyy6polkjvu22eau4a2m
  Reads     : VISIT:a-V1
  Writes    : VISIT:a-V1
  Endorsed  : HospitalMSP
  Response  : 200 OK
└─────────────────────────────────────────────────────────────────────────────
```

```
Block #59
prev_hash : 9N9ZBNcDIVK+IbAv7ySc...
data_hash : V7Ih3zqJhsLFXC06nB4Q...
txns      : 2

Tx #0
TxID      : ba8e03cc10591b6b7bacdd04bf8c4f98e08f28c2322a9e8faa1a17187200a935
Time      : 2026-04-20T19:31:45.703Z
Creator   : HospitalMSP  ()
Function  : AccessContract:LogAccess  PAT-2026-001 visits
Reads     : ACCESS:PAT-2026-001
Writes    : ACCESS:PAT-2026-001
Endorsed  : HospitalMSP
Response  : 200 OK

Tx #1
TxID      : d70e2b1acd9b5937714b85af79660235555ff2a5b89561397eb41cb964b37cba
Time      : 2026-04-20T19:31:45.702Z
Creator   : HospitalMSP  ()
Function  : AccessContract:LogAccess  PAT-2026-001 visits
Reads     : ACCESS:PAT-2026-001
Writes    : ACCESS:PAT-2026-001
Endorsed  : HospitalMSP
Response  : 200 OK
```

Novel Contributions

① Admin-Controlled Staff Registration with Dual-Store Identity Management

Admin-Controlled Staff Registration

Admin Portal

Staff.jsx exposes POST /auth/users restricted to role === 'admin' for registering doctors, nurses, pharmacists & record officers.

Two-Phase Registration

Atomic operation: (1) credential creation in users.json with bcrypt-hashed password, and (2) Hyperledger Fabric CA enrollment. Both must succeed.

Peer Mapping

Doctors → peer1; Nurses, Pharmacists, MedRecord → peer2; Other roles → peer0. Org topology enforced at identity creation.

JWT Tokens

Issued post-login with { userId, role, mspId, peer }. Peer routing for all Fabric gateway calls is locked to the registered role.

POST /auth/users — Registration Endpoint

01

Input Validation

express-validator: username (trimmed), password, role required. Admin role check gates entire handler.

02

Duplicate Prevention

USERS[username] check before write prevents re-registration collisions.

03

Password Security

bcrypt.hash(password, 10) before writing to users.json. Plaintext password is never persisted.

04

Fabric CA Enrollment

execFile('bash', [ENROLL_SCRIPT], { timeout: 30000 }) invokes enrollment shell script asynchronously with 30s timeout.

05

Rollback on Failure

On enrollment failure: re-read users.json → delete new user → re-write file. Keeps app credential store & Fabric CA in sync.

06

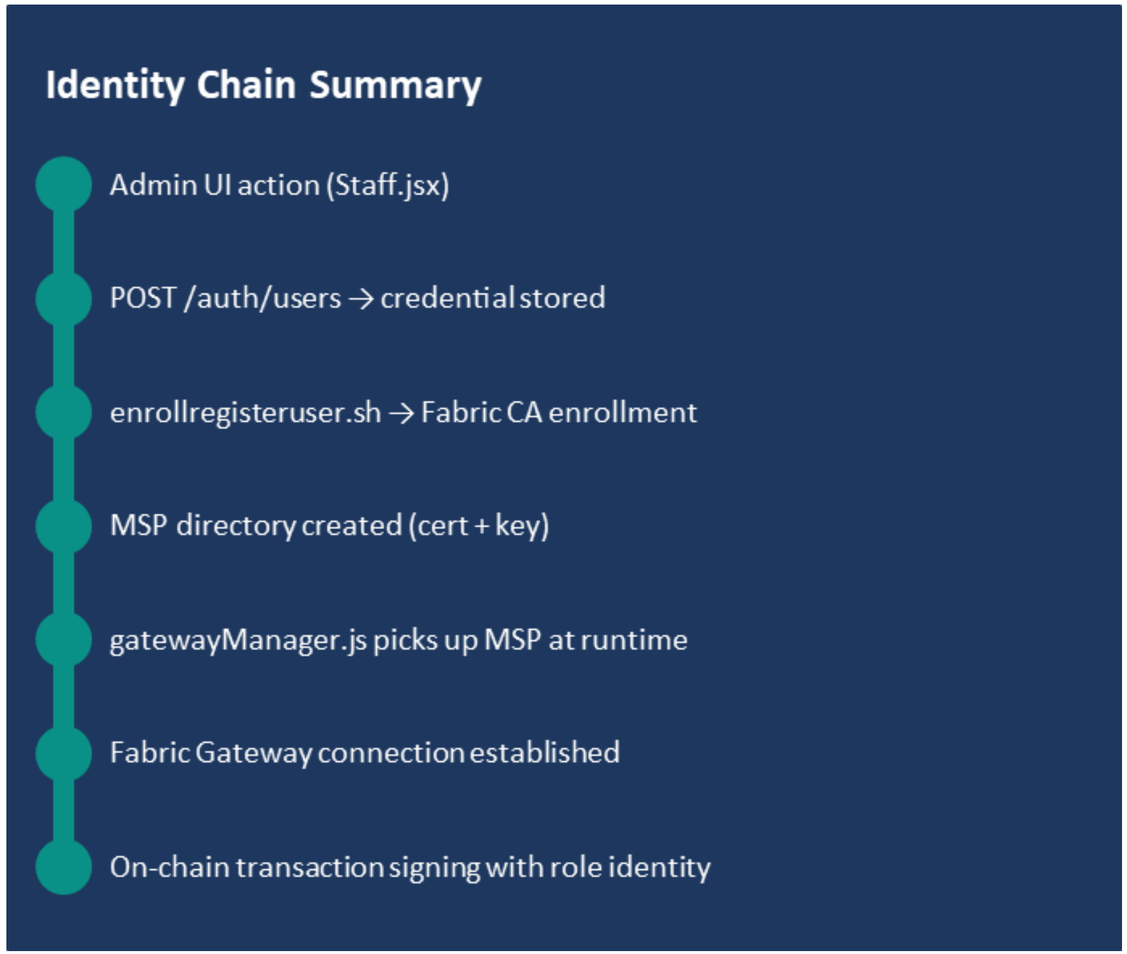
Success Response

HTTP 201 returned with { username, role, mspId: 'HospitalMSP', peer } on complete success.

MSP Folder Creation via enrollregisteruser.sh

STEP 1	STEP 2	STEP 3	STEP 4
CA Admin Enroll	**Register Identity**	**Enroll & Fetch Cert**	**MSP Config.yaml**
CA admin credentials enrolled into /tmp/fabric-ca-temp. Obtains short-lived admin context for registration authorization.	fabric-ca-client register creates the user in Fabric CA database. Role attribute embedded as role=<value>:ecert in certificate.	fabric-ca-client enroll fetches signed X.509 cert + private key → writes to organizations/peerOrganizations/<org>/users/<username>/msp/	config.yaml generated with NodeOUs enabled. Maps client, peer, admin, orderer OU identifiers to org CA cert for OU-based access control.

End-to-End Identity Lifecycle



OCR-Assisted Medical Record Ingestion & EHR PDF Export

Two offline-data-bridge features integrated into the patient portal

OCR Pipeline

Patient uploads scanned doc → POST /ocr via multer memoryStorage → simultaneous IPFS archive + Tesseract.js recognition

PDF Export

generateEhrPdf() constructs structured PDF client-side using jsPDF + jspdf-autotable — no server round-trip

Auth Context

fabricContext middleware validates patient JWT + Fabric identity before any OCR operation is invoked

SERVER-SIDE OCR PIPELINE

patient-api/src/routes/ocr.js

01

multer memoryStorage

File buffered in-memory; no temp file written to disk — minimal attack surface.

02

IPFS Upload First

uploadFile(req.file.buffer, filename) pins the source image to Kubo; CID returned as immutable audit trail before OCR runs.

03

Tesseract.js Recognition

Tesseract.recognize(buffer, 'eng', {...}) runs on the same in-memory buffer using bundled eng.traineddata (~5 MB model).

04

extractAttributes(text)

Six regex patterns: patient name, date, hospital, diagnosis (/(?:Diagnosis|Findings|Impression)\s*:\s*/i), treatment, doctor + generic date fallback.

RESPONSE PAYLOAD

text

Raw OCR string output

attributes

name, date, hospital,
diagnosis, treatment, doctor

sourceCid

IPFS CID of source image

filename

Original upload filename

SLIDE 3 · OCR → BLOCKCHAIN DATA FLOW

Structured OCR Data Committed to Hyperledger Fabric EHR

Patient Reviews OCR Results → **POST /ehr/ medical-history** → **sourceCid Tied to Entry** → **Chaincode Stores Fields**

IPFS Audit Trail

sourceCid permanently ties each EHR entry to its IPFS-pinned source document. Any future audit can fetch the original image by CID.

Structured Fields

attributes object (name, diagnosis, treatment, doctor, date, hospital) stored as structured fields — enables chaincode-level queries.

Two-Stage UX

ocrModal manages Upload + Preview → explicit Save stages, preventing accidental writes of bad OCR output to the immutable ledger.

sourceType: 'ocr'

Entry tagged so downstream consumers and audit tools know the data originated from OCR digitization, not manual entry or direct EHR sync.

SLIDE 4 · EHR PDF EXPORT

Offline-Ready EHR Export via jsPDF + jspdf-autotable

CLIENT-SIDE ONLY

PDF SECTIONS

Section	Fields
Personal Details (4-col)	Name, DOB, Blood Type, ID
Emergency Contact	Name, Relation, Phone
Allergies	Substance / Reaction / Severity
Chronic Conditions	Condition / Status
Ongoing Medications	Name / Dose / Frequency
Immunizations	Vaccine / Date

No Server Round-Trip

generateEhrPdf() works entirely in the browser. EHR data never passes through an intermediate server during export.

Robust Null Handling

typeof item === 'string' fallback + per-item try/catch prevent crashes on partially populated EHR records.

Footer & Filename

Every page stamped with page number + "secured by Blockchain". File saved as Health_Record_<patientId>.pdf.

Security: PDF generated & downloaded entirely in the browser — EHR data from Fabric/IPFS does not pass through any intermediate server during export

Thank You

Decentralized EHR · Hyperledger Fabric · IPFS · Consent Management · Aggregate Signatures

github / project repo · NIT Warangal · MTech CSE Minor Project